

Reviewer Pack — Hyperbolic Volume of Constraint Graphs Bounds SAT Clause Den...

Entry 480b666a471f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 22:06:55 UTC

Hyperbolic Volume of Constraint Graphs Bounds SAT Clause Density

Entry ID: 480b666a471f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 22:06:55 UTC

1. Conjecture

Field A (mathematical branch): Riemannian Geometry and Geometric Measure Theory **Field B** (complexity object): SAT (Satisfiability)

Statement:

For any k -CNF instance with n variables, let G be the constraint graph of the instance. Define the hyperbolic volume of G as the infimum of volumes of all isometric embeddings of G into a complete Riemannian manifold of constant positive curvature. Then, for any constant $c > 0$, the hyperbolic volume of G is upper-bounded by n^{1+c} .

Rationale (proposer's reasoning):

Hyperbolic geometry has been used to study complexity measures in other contexts, such as communication complexity and quantum computation. The use of hyperbolic volume could potentially expose structural properties of constraint graphs that are not apparent in Euclidean or spherical geometries.

Taxonomy category: RIEMANNIAN_GEOMETRY_SAT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e6212008fcefadc8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all k -CNF instances with n variables, the hyperbolic volume of the constraint graph G is upper-bounded by n^{1+c} with a constant $c > 0$ and a support fraction ≥ 0.8 across at least 30 random seeds, AND there are no seeds where the hyperbolic volume exceeds n^{1+c} .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "hyperbolic volume" AND "constraint graph" AND SAT" - "Riemannian geometry" AND geometric measure theory AND satisfiability problem" - "CNF instance" AND Riemannian manifold AND upper bound on clause density"

Top relevant hits considered: - [http://arxiv.org/abs/1206.3093v1] Sub-riemannian geometry from intrinsic viewpoint - [http://arxiv.org/abs/1301.6370v1] The rolling problem: overview and challenges - [http://arxiv.org/abs/hep-th/0612128v2] Riemannian Geometry of Noncommutative Surfaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, clause_density):
        num_clauses = int(clause_density * n * (n - 1))
        clauses = []
        literals = list(range(1, n + 1)) + [-i for i in range(1, n + 1)]

        while len(clauses) < num_clauses:
            clause = random.sample(literals, 2)
            if clause[0] not in clauses and -clause[0] not in clauses and clause[1] not in clauses and -clause[1] not in clauses:
                clauses.append(clause)

        return clauses

    def construct_constraint_graph(clauses):
        n = len(clauses) // (n - 1)
        graph = [[0] * n for _ in range(n)]

        for clause in clauses:
            lit1, lit2 = abs(clause[0]) - 1, abs(clause[1]) - 1
            if 0 <= lit1 < n and 0 <= lit2 < n:
                graph[lit1][lit2] = 1
                graph[lit2][lit1] = 1
```

```

    return graph

def isometric_embedding(graph):
    # Simple heuristic embedding for demonstration purposes
    n = len(graph)
    embedding = {}

    for i in range(n):
        embedding[i] = (random.random(), random.random())

    return embedding

def hyperbolic_volume(embedding):
    n = len(embedding)
    volume = 0

    for i in range(n):
        for j in range(i + 1, n):
            dist = math.sqrt((embedding[i][0] - embedding[j][0]) ** 2 + (embedding[i][1] - embedding[j][1]) ** 2)
            if dist > 0:
                volume += math.log(dist)

    return volume

def check_bound(volume, n, c):
    return volume <= n ** (1 + c)

results = []
for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5): # Ensure at least 30 instances per seed
        clause_density = random.uniform(0.1, 0.9)
        clauses = generate_k_cnf(n, clause_density)
        graph = construct_constraint_graph(clauses)
        embedding = isometric_embedding(graph)
        volume = hyperbolic_volume(embedding)

        results.append({
            "n": n,
            "clause_density": clause_density,
            "volume": volume
        })

total_volumes = sum(result["volume"] for result in results)
mean_volume = Fraction(total_volumes, len(results))
std_volume = math.sqrt(sum((result["volume"] - mean_volume) ** 2 for result in results) / len(results))

c = 0.1
support_fraction = sum(1 for result in results if check_bound(result["volume"], result["n"], c)) / len(results)

conjecture_holds = support_fraction >= 0.8
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Hyperbolic Volume",
    "metric_value": float(mean_volume),
    "instances_tested": len(results),
}

```

```

        "n_max": max(result["n"] for result in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_volumes = sum(result["metric_value"] for result in results)
    mean_volume = Fraction(total_volumes, len(results))
    std_volume = math.sqrt(sum((result["metric_value"] - mean_volume) ** 2 for result in results))

    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_volume} std={std_volume} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9c797c69.py", line 111, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9c797c69.py", line 75, in run_trial
    graph = construct_constraint_graph(clauses)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9c797c69.py", line 34, in construct_constraint_graph
    n = len(clauses) // (n - 1)
        ^
UnboundLocalError: cannot access local variable 'n' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Debug and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14891
2	preregistration	ollama_remote	glm4:latest	0	0	9620
3	novelty	ollama_remote	glm4:latest	0	0	11120
4	novelty	ollama_remote	glm4:latest	0	0	9466
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12142
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10592
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31131
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12919
9	judge	ollama_remote	glm4:latest	0	0	8641

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120523 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/480b666a471f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/480b666a471f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/480b666a471f.tar.gz` (if generated)